CSCI 441/541 · APPLIED DEEP LEARNING · ASSIGNMENT 1 + PROJECT 1 REPORT

jb-"type"

Student: Jordan.brown.1@und.edu · **Generated:** 2026-10-18

Overview

I trained a linear softmax classifier and a two-hidden-layer ReLU MLP to recognize the 11 numeral characters. Both models turn each 64×64 grayscale image into 16×16 features by averaging 4×4 pixel blocks, then invert and scale the values to roughly [−1, 1]. The MLP has hidden widths 128 and 64 and produces 11 logits. The main lesson was that nonlinear hidden layers can combine stroke features that a single linear decision rule struggles to separate.

Dataset & collection

Copybook coverage

Reconstructed from the exported labels.json · 442 original captures · 22/22 cells meet the 3-capture minimum

	fontA	fontB
Zero	20	20
One	20	20
Two	20	20
Three	20	20
Four	21	20
Five	20	20
Six	20	20
Seven	20	20
Eight	20	20
Nine	21	20
Ten	20	20

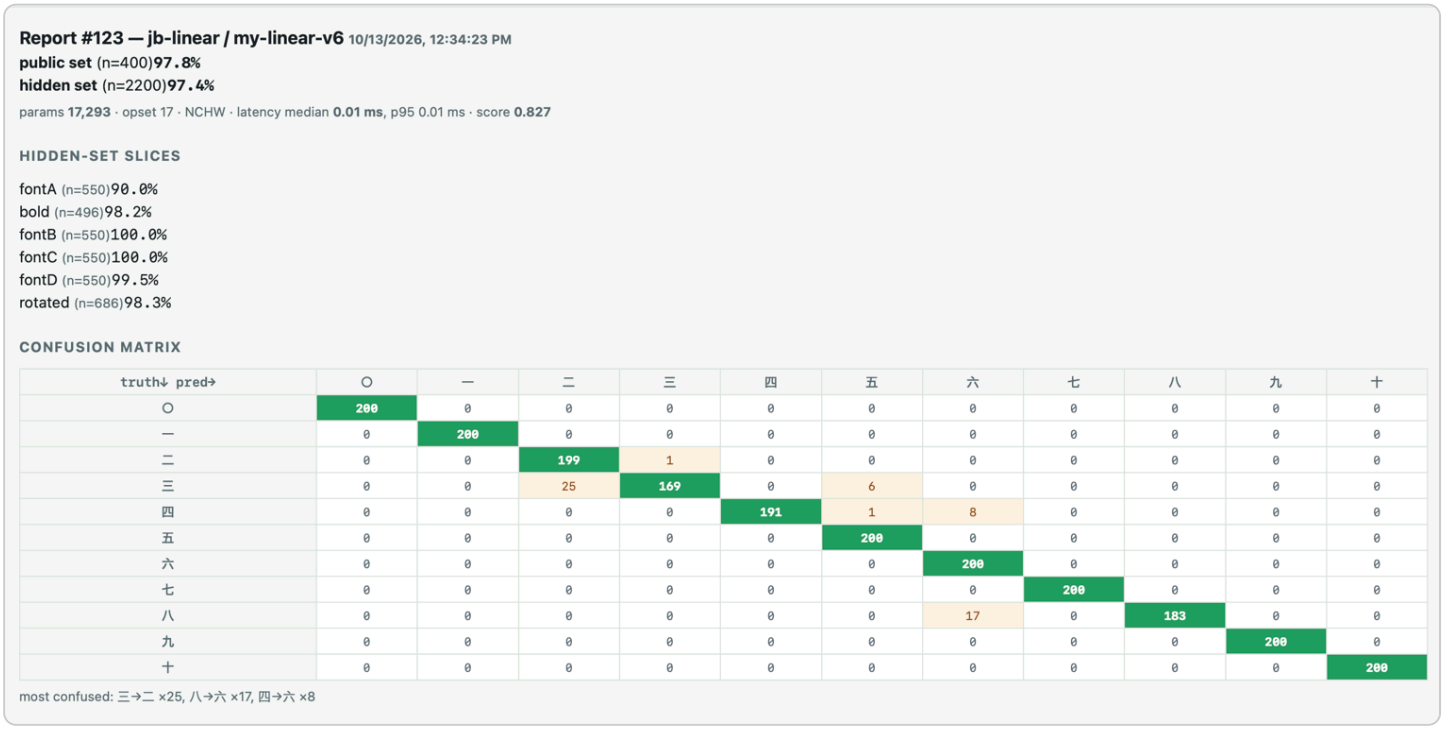
Each trace was exported at three stroke widths (−3%, original, +3%), yielding 1,326 images.

How many raw captures per character did you collect, and how did you keep the classes balanced? Any capture you redid and why?

I traced 442 original captures from the two guide fonts: 40 per character, except four and nine with 41 each. Every character × font cell has 20 captures, except fontA for four and nine with 21, so the classes are nearly balanced and every cell exceeds the required three. The Copybook export rendered each capture at three stroke widths (−3%, original, +3%), producing 1,326 exported images; adding 1,000 augmentations per exported image produced 1,327,326 images. I did not redo any captures. I unfortunately lost the original copybook, so I made a similar thing instead

Assignment — linear softmax baseline

Linear evidence block 1



Report #123 — jb-linear / my-linear-v6 10/13/2026, 12:34:23 PM
public set (n=400) **97.8%**
hidden set (n=2200) **97.4%**
params **17,293** · opset 17 · NCHW · latency median **0.01 ms**, p95 0.01 ms · score **0.827**

HIDDEN-SET SLICES

fontA (n=550) **90.0%**
bold (n=496) **98.2%**
fontB (n=550) **100.0%**
fontC (n=550) **100.0%**
fontD (n=550) **99.5%**
rotated (n=686) **98.3%**

CONFUSION MATRIX

truth↓ pred→	○	一	二	三	四	五	六	七	八	九	十
○	200	0	0	0	0	0	0	0	0	0	0
一	0	200	0	0	0	0	0	0	0	0	0
二	0	0	199	1	0	0	0	0	0	0	0
三	0	0	25	169	0	6	0	0	0	0	0
四	0	0	0	0	191	1	8	0	0	0	0
五	0	0	0	0	0	200	0	0	0	0	0
六	0	0	0	0	0	0	200	0	0	0	0
七	0	0	0	0	0	0	0	200	0	0	0
八	0	0	0	0	0	0	17	0	183	0	0
九	0	0	0	0	0	0	0	0	0	200	0
十	0	0	0	0	0	0	0	0	0	0	200

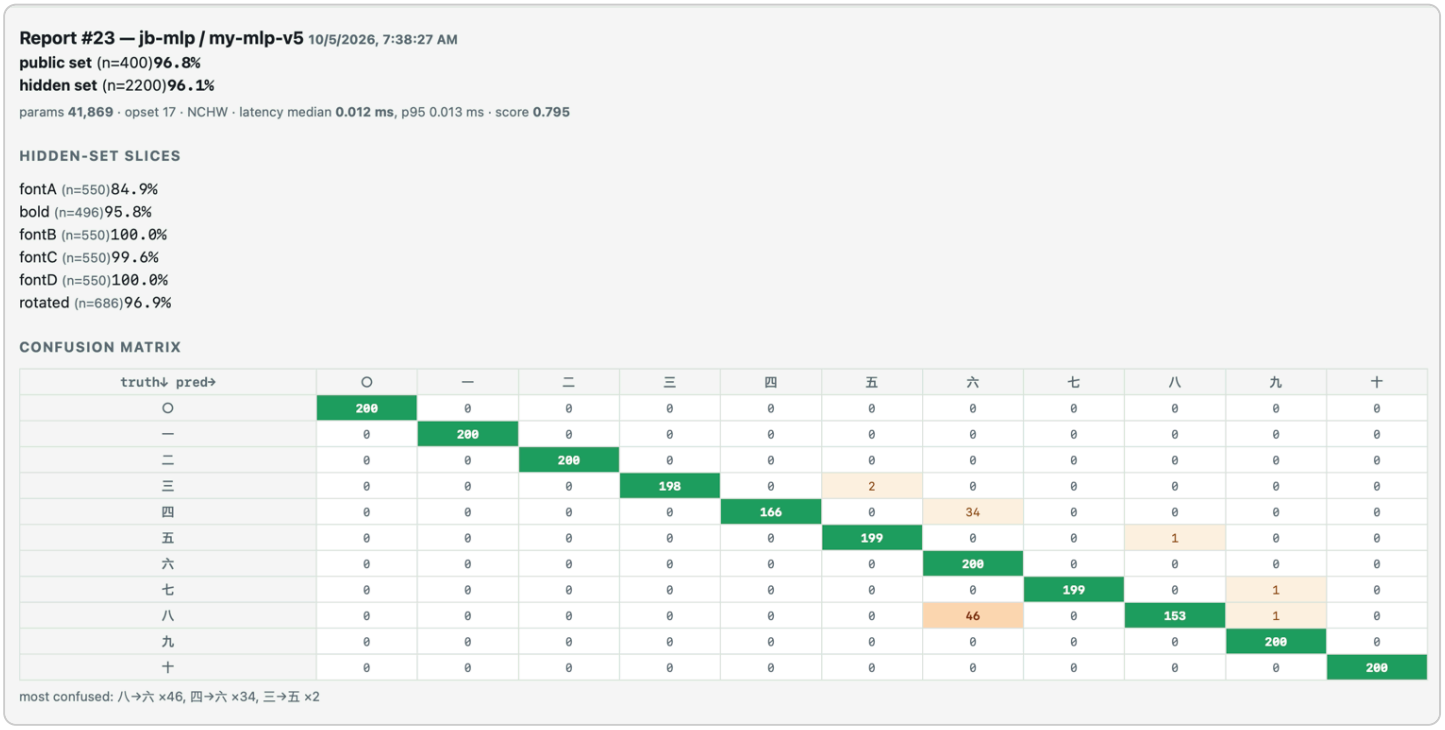
most confused: 三→二 ×25, 八→六 ×17, 四→六 ×8

Description

The model achieved 97.8% public accuracy and 97.4% hidden accuracy, with 17,293 parameters, a median inference latency of 0.01 ms, and an overall score of 0.827. Errors were concentrated in Font A and visually similar classes, suggesting limited generalization across font variations.

Project — the MLP

Experiment block 1



Report #23 — jb-mlp / my-mlp-v5 10/5/2026, 7:38:27 AM
public set (n=400)**96.8%**
hidden set (n=2200)**96.1%**
params **41,869** · opset 17 · NCHW · latency median **0.012 ms**, p95 0.013 ms · score **0.795**

HIDDEN-SET SLICES

fontA (n=550)84.9%
bold (n=496)95.8%
fontB (n=550)100.0%
fontC (n=550)99.6%
fontD (n=550)100.0%
rotated (n=686)96.9%

CONFUSION MATRIX

truth↓ pred→	○	一	二	三	四	五	六	七	八	九	十
○	200	0	0	0	0	0	0	0	0	0	0
一	0	200	0	0	0	0	0	0	0	0	0
二	0	0	200	0	0	0	0	0	0	0	0
三	0	0	0	198	0	2	0	0	0	0	0
四	0	0	0	0	166	0	34	0	0	0	0
五	0	0	0	0	0	199	0	0	1	0	0
六	0	0	0	0	0	0	200	0	0	0	0
七	0	0	0	0	0	0	0	199	0	1	0
八	0	0	0	0	0	0	46	0	153	1	0
九	0	0	0	0	0	0	0	0	0	200	0
十	0	0	0	0	0	0	0	0	0	0	200

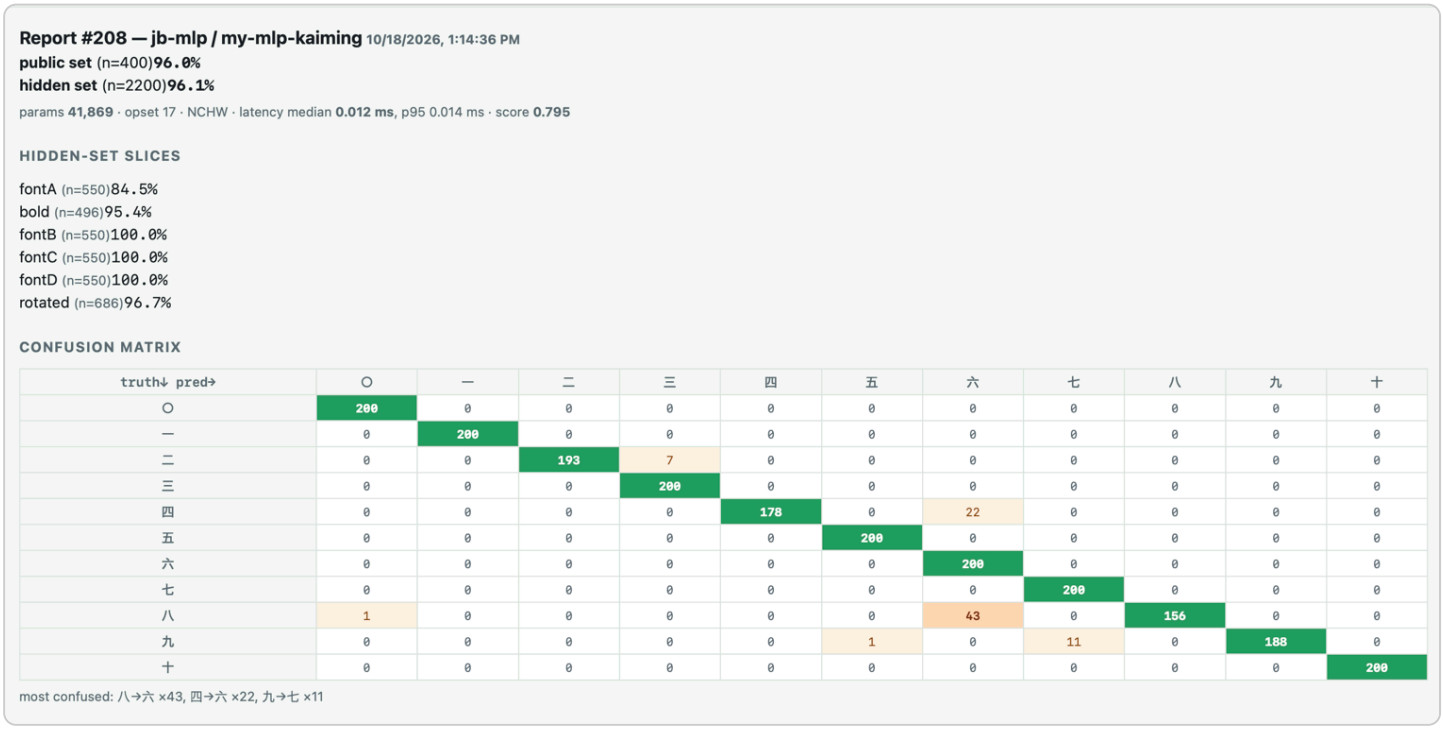
most confused: 八→六 ×46, 四→六 ×34, 三→五 ×2

baseline mlp

Description

The baseline MLP achieved 96.8% public accuracy and 96.1% hidden accuracy, with 41,869 parameters, a median inference latency of 0.012 ms, and an overall score of 0.795. Errors were concentrated in Font A, which achieved 84.9% accuracy, indicating limited generalization across font variations.

Experiment block 2



Report #208 — jb-mlp / my-mlp-kaiming 10/18/2026, 1:14:36 PM
public set (n=400)**96.0%**
hidden set (n=2200)**96.1%**
params **41,869** · opset 17 · NCHW · latency median **0.012 ms**, p95 0.014 ms · score **0.795**

HIDDEN-SET SLICES

fontA (n=550)84.5%
bold (n=496)95.4%
fontB (n=550)100.0%
fontC (n=550)100.0%
fontD (n=550)100.0%
rotated (n=686)96.7%

CONFUSION MATRIX

truth↓ pred→	○	一	二	三	四	五	六	七	八	九	十
○	200	0	0	0	0	0	0	0	0	0	0
一	0	200	0	0	0	0	0	0	0	0	0
二	0	0	193	7	0	0	0	0	0	0	0
三	0	0	0	200	0	0	0	0	0	0	0
四	0	0	0	0	178	0	22	0	0	0	0
五	0	0	0	0	0	200	0	0	0	0	0
六	0	0	0	0	0	0	200	0	0	0	0
七	0	0	0	0	0	0	0	200	0	0	0
八	1	0	0	0	0	0	43	0	156	0	0
九	0	0	0	0	0	1	0	11	0	188	0
十	0	0	0	0	0	0	0	0	0	0	200

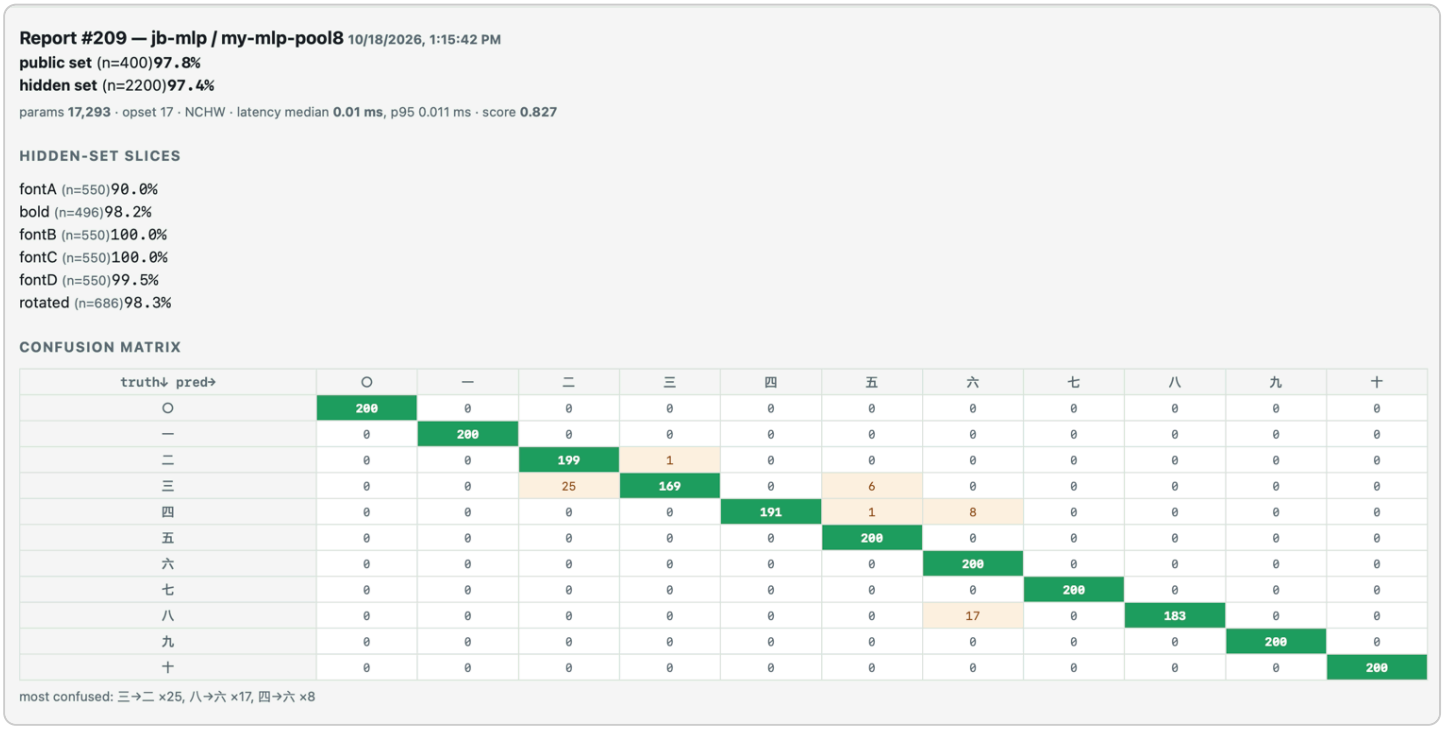
most confused: 八→六 ×43, 四→六 ×22, 九→七 ×11

kaiming mlp

Description

The MLP with Kaiming initialization achieved 96.0% public accuracy and 96.1% hidden accuracy, with 41,869 parameters, a median inference latency of 0.012 ms, and an overall score of 0.795. Hidden accuracy and overall score matched the baseline, while Font A accuracy remained low at 84.5%, suggesting that initialization alone did not improve generalization across font variations.

Experiment block 3



Report #209 — jb-mlp / my-mlp-pool8 10/18/2026, 1:15:42 PM
public set (n=400)**97.8%**
hidden set (n=2200)**97.4%**
params **17,293** · opset 17 · NCHW · latency median **0.01 ms**, p95 0.011 ms · score **0.827**

HIDDEN-SET SLICES

fontA (n=550)**90.0%**
bold (n=496)**98.2%**
fontB (n=550)**100.0%**
fontC (n=550)**100.0%**
fontD (n=550)**99.5%**
rotated (n=686)**98.3%**

CONFUSION MATRIX

truth↓ pred→	○	一	二	三	四	五	六	七	八	九	十
○	200	0	0	0	0	0	0	0	0	0	0
一	0	200	0	0	0	0	0	0	0	0	0
二	0	0	199	1	0	0	0	0	0	0	0
三	0	0	25	169	0	6	0	0	0	0	0
四	0	0	0	0	191	1	8	0	0	0	0
五	0	0	0	0	0	200	0	0	0	0	0
六	0	0	0	0	0	0	200	0	0	0	0
七	0	0	0	0	0	0	0	200	0	0	0
八	0	0	0	0	0	0	17	0	183	0	0
九	0	0	0	0	0	0	0	0	0	200	0
十	0	0	0	0	0	0	0	0	0	0	200

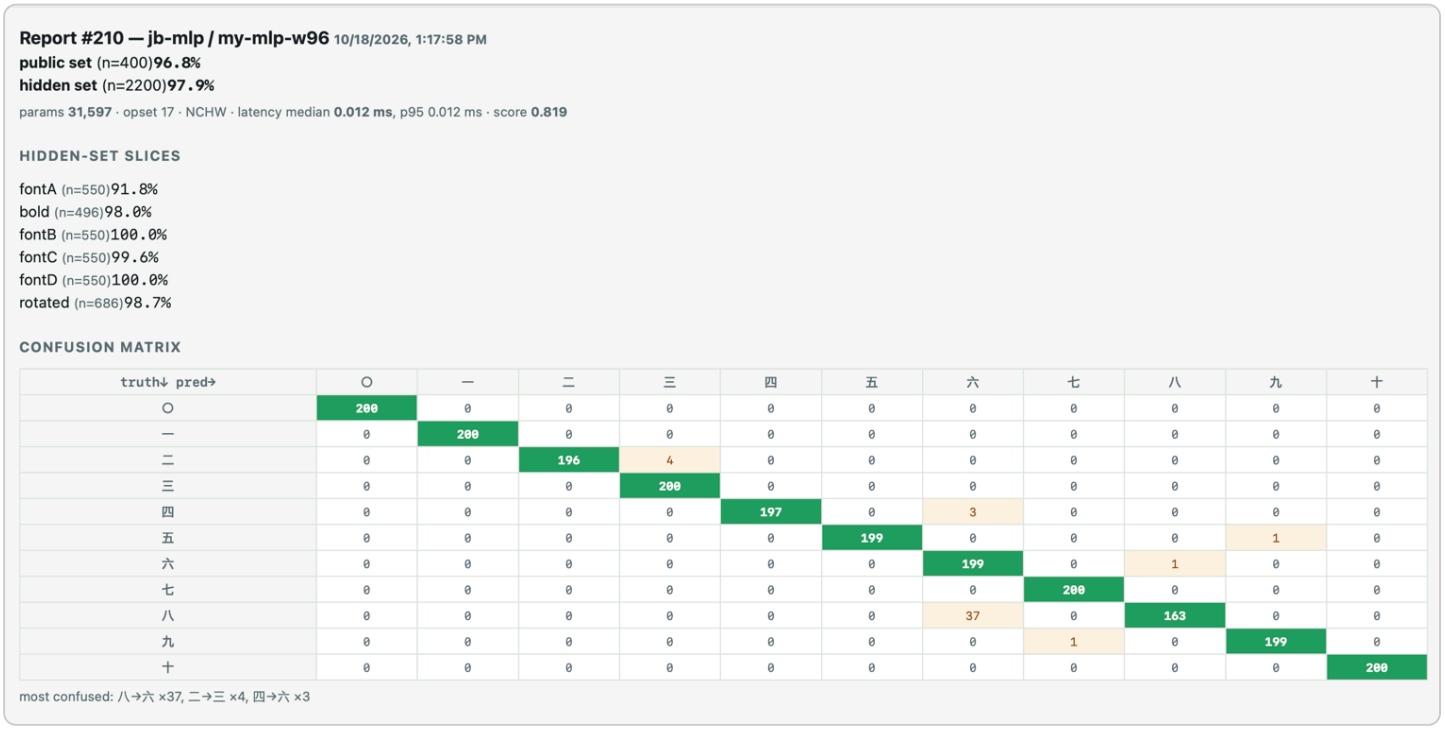
most confused: 三→二 ×25, 八→六 ×17, 四→六 ×8

pool 8 mlp

Description

The Pool8 MLP achieved 97.8% public accuracy and 97.4% hidden accuracy, with 17,293 parameters, a median inference latency of 0.010 ms, and an overall score of 0.827. Compared with the baseline, it improved hidden accuracy by 1.3 percentage points while reducing model size and latency. Font A accuracy increased to 90.0%, although it remained the primary source of classification errors.

Experiment block 4



Report #210 — jb-mlp / my-mlp-w96 10/18/2026, 1:17:58 PM
public set (n=400)**96.8%**
hidden set (n=2200)**97.9%**
params **31,597** · opset 17 · NCHW · latency median **0.012 ms**, p95 0.012 ms · score **0.819**

HIDDEN-SET SLICES

fontA (n=550)**91.8%**
bold (n=496)**98.0%**
fontB (n=550)**100.0%**
fontC (n=550)**99.6%**
fontD (n=550)**100.0%**
rotated (n=686)**98.7%**

CONFUSION MATRIX

truth↓ pred→	○	一	二	三	四	五	六	七	八	九	十
○	200	0	0	0	0	0	0	0	0	0	0
一	0	200	0	0	0	0	0	0	0	0	0
二	0	0	196	4	0	0	0	0	0	0	0
三	0	0	0	200	0	0	0	0	0	0	0
四	0	0	0	0	197	0	3	0	0	0	0
五	0	0	0	0	0	199	0	0	0	1	0
六	0	0	0	0	0	0	199	0	1	0	0
七	0	0	0	0	0	0	0	200	0	0	0
八	0	0	0	0	0	0	37	0	163	0	0
九	0	0	0	0	0	0	0	1	0	199	0
十	0	0	0	0	0	0	0	0	0	0	200

most confused: 八→六 ×37, 二→三 ×4, 四→六 ×3

width 96 mlp

Description

The width 96 MLP achieved 96.8% public accuracy and 97.9% hidden accuracy, with 31,597 parameters, a median inference latency of 0.012 ms, and an overall score of 0.819. Compared with the baseline, it improved hidden accuracy by 1.8 percentage points while reducing model size. Font A accuracy increased to 91.8%, although it remained the primary source of classification errors.

Evaluation Site results

Submission IDs

Report #209 — jb-mlp / my-mlp-pool8 10/18/2026, 1:15:42 PM

public set (n=400)**97.8%**

hidden set (n=2200)**97.4%**

params **17,293** · opset 17 · NCHW · latency median **0.01 ms**, p95 0.011 ms · score **0.827**

HIDDEN-SET SLICES

fontA (n=550)90.0%
bold (n=496)98.2%
fontB (n=550)100.0%
fontC (n=550)100.0%
fontD (n=550)99.5%
rotated (n=686)98.3%

List both submission IDs (linear and MLP) and their scores.

The MLP model, my-mlp-pool8 (report #209), and the linear submission, my-linear-v6 (report #123), both achieved 97.8% public accuracy on 400 samples and 97.4% hidden accuracy on 2,200 samples, with 17,293 parameters and a score of 0.827. Both had a median latency of 0.010 ms; their 95th-percentile latencies were 0.011 ms for the MLP and 0.010 ms for the linear submission. Both reported hidden-set accuracies of 90.0% for fontA, 100.0% for fontB, 100.0% for fontC, 99.5% for fontD, 98.2% for bold images, and 98.3% for rotated images, using ONNX opset 17 and the NCHW input layout.

Hidden-dataset results & analysis

Report #209 — jb-mlp / my-mlp-pool8 10/18/2026, 1:15:42 PM
public set (n=400)**97.8%**
hidden set (n=2200)**97.4%**

params **17,293** · opset 17 · NCHW · latency median **0.01 ms**, p95 0.011 ms · score **0.827**

HIDDEN-SET SLICES

fontA (n=550)**90.0%**
bold (n=496)**98.2%**
fontB (n=550)**100.0%**
fontC (n=550)**100.0%**
fontD (n=550)**99.5%**
rotated (n=686)**98.3%**

Analysis (graded): which font slice or confusion pair is weakest, and why do you think that happens? What would you try next?

The weakest font slice was fontA at 90.0%, compared with 99.5–100% for the other fonts. The largest confusion was three → two (25/200), followed by eight → six (17/200). One possible explanation is that 8×8 pooling blurs distinguishing stroke details, such as the extra horizontal stroke in three. Next, I would inspect the misclassified images, add targeted examples with varied stroke thickness and spacing, and compare pooling sizes under identical training conditions to see whether preserving finer details reduces these errors.

Comparison & conclusion

Linear vs MLP on the hidden set (accuracy, parameters, latency)?

The linear and MLP submissions both achieved 97.4% hidden-set accuracy with 17,293 parameters and a median latency of 0.010 ms. The MLP's 95th-percentile latency was 0.011 ms, compared with 0.010 ms for the linear model. Both achieved identical slice accuracies: 90.0% on fontA, 100.0% on fontB and fontC, 99.5% on fontD, 98.2% on bold images, and 98.3% on rotated images. Based on these results, the MLP used no extra parameters and provided no accuracy improvement on any reported slice.

Most helpful change, and what did not help?

The single change that helped most was increasing the variety of training augmentations. Adding stronger rotations, translations, scaling, shear, and elastic distortions gave the model more varied examples and likely helped it recognize characters beyond the original captures. Finer pooling did not improve hidden accuracy in the earlier experiment, suggesting that preserving more detail alone was not enough to improve generalization. Switching from Xavier to Kaiming initialization also did not improve local validation accuracy. These experiments suggest that broader training variation was more useful than the pooling and initialization changes I tried.

Appendix A — sample_mlp.py

```
# CSCI 441/541 — Assignment 1 + Project 1 · guided scaffold
#
# ONE file, TWO deliverables:
#   PART A (Assignment): linear-softmax baseline  -> linear_model.onnx
#   PART B (Project):    your own MLP             -> mlp_model.onnx
#
# Everything marked  TODO  is yours to write. Everything else is provided:
# data loading, the lecture-recipe softmax, evaluation, and the ONNX export
# for Part A. Unfinished TODOs raise with a hint — run top to bottom and let
# the asserts tell you what is still missing.
#
# Your toolbox is exactly the three decks taught so far:
#   Lecture 7 — logits, softmax, cross-entropy, the (P - Y) gradient
#   Lecture 8 — MLPs, activations, from-scratch SGD updates, torch loops
#   Lecture 9 — nn.Module classes, parameters(), .apply initialization,
#               the four-step training step, save/load
# One idea is NEW here and explained inline: the image representation
# (TODO-B1). The ONNX export is PROVIDED in full — packaging is not
# assessed. Everything else reuses the lectures verbatim. The Project
# grade rewards what YOU design inside the 50,000-parameter budget.

import io
import json
import zipfile
import numpy as np
from PIL import Image

# Colab doesn't ship `onnx` — install it silently if missing
import importlib.util, subprocess, sys
if importlib.util.find_spec("onnx") is None:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "onnx"])

# ---------------------------------------------------------------- shared ---

GLYPHS = "〇一二三四五六七八九" + "十"      # class order = label ids 0..10
```

```
N_CLASSES = 11
INPUT_SHAPE = (1, 1, 64, 64)               # model contract: NCHW float32 [0,1]


def load_copybook(dataset_path):
    """Accepts a Copybook export zip or an unpacked directory.

    Returns (images float32 [n,64,64] in [0,1], labels int64 [n])."""
    if dataset_path.endswith(".zip"):
        zf = zipfile.ZipFile(dataset_path)
        meta = json.loads(zf.read("labels.json") if "labels.json" in zf.namelist()
                          else zf.read("manifest.json"))
        if isinstance(meta, dict):                          # copybook layout
            meta = meta.get("labels") or meta.get("shots")
        shots = []
        for m in meta:                                      # manifest order
            name = f"shots/{m['file']}"
            img = Image.open(io.BytesIO(zf.read(name))).convert("L")
            shots.append(np.asarray(img, np.float32) / 255.0)
        return (np.stack(shots),
                np.array([m["label"] for m in meta], dtype=np.int64))
    raise SystemExit("point --dataset at your Copybook export zip")


def softmax(z):
    """Stable softmax (lecture 7 recipe). Provided."""
    z = z - z.max(axis=1, keepdims=True)
    e = np.exp(z)
    return e / e.sum(axis=1, keepdims=True)


# ============================================================== PART A ===
# Assignment: linear softmax.  The affine map IS the whole model — you
# assemble forward, gradient, and update; the export below is provided.

D_A = 16 * 16          # representation width: 16x16 average-pooled grayscale


def part_a_features(images):
    """64x64 -> 16x16 average pool, ink positive, scaled to [-1,1]."""
    x = images.reshape(-1, 16, 4, 16, 4).mean(axis=(2, 4))
    return ((0.5 - x) * 2.0).reshape(len(images), -1)   # ink dark: invert


def part_a_train(X, y, epochs=20, lr=0.05, batch=512):
    n, d = X.shape
    W = np.zeros((d, N_CLASSES), np.float32)    # zero init: sanity check below
    b = np.zeros(N_CLASSES, np.float32)

    # ---- sanity: zero weights must give loss = ln(11) ----------------------
```

```
    n_check = min(64, n)   # works even with a tiny first dataset
    p0 = softmax(X[:n_check] @ W + b)
    loss0 = -np.log(p0[np.arange(n_check), y[:n_check]] + 1e-12).mean()
    assert abs(loss0 - np.log(N_CLASSES)) < 1e-3, "softmax/CE sanity failed"

    for ep in range(epochs):
        perm = np.random.default_rng(ep).permutation(n)
        for i in range(0, n, batch):
            j = perm[i:i + batch]
            Xb, yb = X[j], y[j]

            # >>> FILL A1
            logits = Xb @ W + b
            # <<< FILL A1

            # >>> FILL A2
            G = softmax(logits)
            G[np.arange(len(j)), yb] -= 1.0
            G /= len(j)
            # <<< FILL A2

            # >>> FILL A3
            dW = Xb.T @ G
            db = G.sum(axis=0)
            W -= lr * dW
            b -= lr * db
            # <<< FILL A3

        if ep % 20 == 0 or ep == epochs - 1:
            acc = (softmax(X @ W + b).argmax(1) == y).mean()
            print(f"[A] epoch {ep:3d}  train acc {acc:.3f}")
    return W, b


# Provided: pack the trained affine map into the contract graph
# (avg-pool 4x4 -> invert/scale -> flatten -> Gemm).
def export_linear(W, b, path="linear_model.onnx"):
    import onnx
    from onnx import helper, TensorProto

    def tensor(name, arr):
        return helper.make_tensor(name, TensorProto.FLOAT, arr.shape,
                                  arr.astype(np.float32).tobytes(), raw=True)

    # (provided) parameter-free preprocessing: AvgPool k=4 then affine
    shape = helper.make_tensor("fshape", TensorProto.INT64, [2],
                               np.array([1, D_A], np.int64).tobytes(), raw=True)
    nodes = [
        helper.make_node("AveragePool", ["image"], ["p"], kernel_shape=[4, 4], strides=[4,
4]),
```

```
        helper.make_node("Constant", [], ["half"],
                         value=tensor("half", np.array(0.5, np.float32))),
        helper.make_node("Sub", ["half", "p"], ["inv"]),          # 0.5 - pool
        helper.make_node("Constant", [], ["two"],
                         value=tensor("two", np.array(2.0, np.float32))),
        helper.make_node("Mul", ["inv", "two"], ["x"]),           # scale
        helper.make_node("Reshape", ["x", "fshape"], ["f"]),      # [1, 256]
        helper.make_node("Gemm", ["f", "Wt", "bt"], ["logits"]),
    ]
    graph = helper.make_graph(
        nodes, "linear_softmax",
        [helper.make_tensor_value_info("image", TensorProto.FLOAT, INPUT_SHAPE)],
        [helper.make_tensor_value_info("logits", TensorProto.FLOAT,
                                       [1, N_CLASSES])],
        [tensor("Wt", W.astype(np.float32)),
         tensor("bt", b), shape])
    model = helper.make_model(graph, opset_imports=[
        helper.make_opsetid("", 17)])
    model.ir_version = 9
    onnx.checker.check_model(model)
    onnx.save(model, path)
    print(f"[A] wrote {path}  ({W.size + b.size} params) — submit for the Assignment")


# ================================================================ PART B ===
# Project: the MLP is YOURS. Representation, architecture, initialization,
# training loop, and export — each a numbered TODO. Budget: <= 50,000
# float parameters in the exported graph (preprocessing is parameter-free
# and free). Conv/RNN ops are banned by the contract; Gemm + activations only.

import torch
from torch import nn
import torch.nn.functional as F

BUDGET = 50_000


# TODO-B1: representation — NEW IDEA, not from the lectures.
#   A 64x64 image is 4,096 numbers; a full-size first layer would blow the
#   budget. Instead, treat each k x k BLOCK of pixels as one number: its
#   average. With k = 4 the image becomes 16 x 16 = 256 numbers (k = 8
#   gives 8 x 8 = 64: cheaper, blurrier — your call, and your experiment).
#   Micro-example for k = 4 on one image:  blocks = gray.reshape(16, 4, 16, 4)
#   ... then average over the two block axes ... then .reshape(len, -1).
#   Remember the ink is DARK (values near 0) on a white (1.0) background,
#   and networks prefer small symmetric inputs.
def part_b_features(images):
    """images [n,64,64] in [0,1]  ->  features [n, d], bounded."""
    # >>> FILL B1
    pooled = images.reshape(-1, 16, 4, 16, 4).mean(axis=(2, 4))
```

```
    return ((0.5 - pooled) * 2.0).reshape(len(images), -1)
    # <<< FILL B1


# TODO-B2: parameter accounting. Every float in the graph counts toward the
#   budget — weights AND biases, for every layer.
def count_params(model):
    # >>> FILL B2
    return sum(p.numel() for p in model.parameters())
    # <<< FILL B2


# TODO-B3: the model. A nn.Module (Lecture 9 pattern):
#   - __init__ declares your layers ONCE (choose widths so count_params
#     stays inside the budget — print it and adjust);
#   - forward(x) takes [1,1,64,64] and MUST do the preprocessing inside
#     the graph BEFORE the first linear layer, so the exported ONNX is
#     self-contained. Port your B1 code to torch: tensors reshape and
#     mean exactly like NumPy arrays (x = x.reshape(1, 16, 4, 16, 4)...);
#     torch.nn.functional.avg_pool2d(x, 4) is the ready-made equivalent.
#   - the hidden activation is your choice (Lecture 8).
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        # >>> FILL B3a
        self.fc1 = nn.Linear(16 * 16, 128)
        self.fc2 = nn.Linear(128, 64)
        self.fc3 = nn.Linear(64, N_CLASSES)
        # <<< FILL B3a

    def forward(self, x):                    # x: [1,1,64,64]
        # >>> FILL B3b
        x = (0.5 - F.avg_pool2d(x, 4)) * 2.0
        x = x.flatten(start_dim=1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        return self.fc3(x)
        # <<< FILL B3b


# TODO-B4: initialization policy via .apply (Lecture 9). Xavier on weights,
#   zeros on biases is the sane default — anything deliberate is fine.
def init_weights(m):
    # >>> FILL B4
    if isinstance(m, nn.Linear):
        nn.init.xavier_uniform_(m.weight)
        nn.init.zeros_(m.bias)
    # <<< FILL B4
```

```
def part_b_train(model, images, y, imgs_val, yval, epochs=25, lr=0.01, batch=256):
    """Feed RAW images [n,1,64,64]: the model preprocesses inside forward,
    exactly like the exported graph will (Lecture 9 encapsulation)."""
    X = torch.from_numpy(images.reshape(-1, 1, 64, 64))
    XvT = torch.from_numpy(imgs_val.reshape(-1, 1, 64, 64))
    yt = torch.from_numpy(y)
    opt = torch.optim.SGD(model.parameters(), lr=lr, momentum=0.9)
    loss_fn = nn.CrossEntropyLoss()
    rng = np.random.default_rng(0)

    for ep in range(epochs):
        model.train()
        perm = rng.permutation(len(X))
        for i in range(0, len(perm), batch):
            j = perm[i:i + batch]

            # >>> FILL B5
            opt.zero_grad()
            loss = loss_fn(model(X[j]), yt[j])
            loss.backward()
            opt.step()
            # <<< FILL B5

        model.eval()
        with torch.no_grad():
            val_acc = (model(XvT).argmax(1)
                       == torch.from_numpy(yval)).float().mean().item()
        if ep % 20 == 0 or ep == epochs - 1:
            print(f"[B] epoch {ep:3d}  val acc {val_acc:.3f}")


# PROVIDED — the ONNX export. Packaging is not assessed; this is the
# model contract made concrete: fixed input [1,1,64,64] named "image",
# output named "logits", opset 17, no dynamic batch axis. Your forward()
# runs inside the exported graph exactly as written.
def export_mlp(model, path="mlp_model.onnx"):
    model.eval()
    # >>> PROVIDED B6
    torch.onnx.export(model, torch.zeros(1, 1, 64, 64), path,
                      input_names=["image"], output_names=["logits"],
                      opset_version=17, dynamo=False)
    # <<< PROVIDED B6


# Large-directory runner. The provided ZIP loader and exports remain unchanged.
from collections import defaultdict
from pathlib import Path
from torch.utils.data import DataLoader, Dataset
from tqdm import tqdm
```

```
class CopybookDirectory(Dataset):
    """Read selected images from an unpacked Copybook dataset on demand."""

    def __init__(self, directory, metadata, indices=None):
        self.shots = Path(directory) / "shots"
        self.metadata = metadata
        self.indices = indices

    def __len__(self):
        return len(self.metadata) if self.indices is None else len(self.indices)

    def __getitem__(self, position):
        index = position if self.indices is None else int(self.indices[position])
        item = self.metadata[index]
        with Image.open(self.shots / item["file"]) as image:
            pixels = np.asarray(image.convert("L"), dtype=np.float32) / 255.0
        return torch.from_numpy(pixels[None]), int(item["label"])


def capture_key(item):
    """Group all augmentations and three stroke widths of one trace."""
    source = item.get("augmented_from", item["file"])
    stem = Path(source).stem
    capture = int(stem) // 3 if stem.isdigit() and "wscale" in item else stem
    return int(item["label"]), item.get("font", "unknown"), capture


def grouped_holdout(metadata, fraction=0.2, seed=42):
    """Choose whole captures within each character/font cell for validation."""
    groups_by_cell = defaultdict(set)
    for item in metadata:
        if "augmented_from" not in item:
            label, font, capture = capture_key(item)
            groups_by_cell[label, font].add(capture)
    rng = np.random.default_rng(seed)
    held_out = set()
    for cell, captures in sorted(groups_by_cell.items()):
        ordered = sorted(captures)
        if len(ordered) < 2:
            raise ValueError(f"character/font cell {cell} needs two captures")
        count = min(len(ordered) - 1, max(1, round(len(ordered) * fraction)))
        for position in rng.choice(len(ordered), size=count, replace=False):
            held_out.add((*cell, ordered[int(position)]))
    train_indices, val_indices = [], []
    for index, item in enumerate(metadata):
        (val_indices if capture_key(item) in held_out else train_indices).append(index)
    return np.asarray(train_indices, np.int64), np.asarray(val_indices, np.int64), held_out
```

```
def linear_validation_features(dataset, batch=512):
    """Cache only pooled validation features, about 1 KiB per image."""
    features = np.empty((len(dataset), D_A), np.float32)
    labels = np.empty(len(dataset), np.int64)
    loader = DataLoader(dataset, batch_size=batch)
    position = 0
    for images, targets in tqdm(loader, desc="[A] val features", unit="batch"):
        end = position + len(targets)
        features[position:end] = part_a_features(images.numpy()[:, 0])
        labels[position:end] = targets.numpy()
        position = end
    return features, labels


def train_linear_directory(dataset, epochs, val=None, lr=0.05, batch=512,
                           schedule_epochs=20):
    """Run linear SGD; use the grouped holdout only when val is supplied."""
    W = np.zeros((D_A, N_CLASSES), np.float32)
    b = np.zeros(N_CLASSES, np.float32)
    first_image, first_label = dataset[0]
    first_feature = part_a_features(first_image.numpy()[None, 0])
    p0 = softmax(first_feature @ W + b)[0, first_label]
    loss0 = -np.log(p0 + 1e-12)
    assert abs(loss0 - np.log(N_CLASSES)) < 1e-3, "softmax/CE sanity failed"
    print(f"[A] initial loss {loss0:.4f} (expected ln(11) = {np.log(N_CLASSES):.4f})")
    val_features = linear_validation_features(val) if val is not None else None
    loader = DataLoader(dataset, batch_size=batch, shuffle=True,
                        generator=torch.Generator().manual_seed(0))
    best_acc, best_loss, best_epoch = -1.0, float("inf"), 1
    for ep in range(epochs):
        rate = lr * (0.2 + 0.8 * (1 + np.cos(np.pi * ep / schedule_epochs)) / 2)
        for images, targets in tqdm(loader, desc=f"[A] epoch {ep + 1}/{epochs}",
                                    unit="batch"):
            Xb = part_a_features(images.numpy()[:, 0])
            yb = targets.numpy()
            G = softmax(Xb @ W + b)
            G[np.arange(len(yb)), yb] -= 1.0
            G /= len(yb)
            W -= rate * (Xb.T @ G)
            b -= rate * G.sum(axis=0)
        if val_features is not None:
            X_val, y_val = val_features
            predictions = softmax(X_val @ W + b)
            acc = float((predictions.argmax(1) == y_val).mean())
            loss = float(-np.log(
                predictions[np.arange(len(y_val)), y_val] + 1e-12
            ).mean())
            print(f"[A] epoch {ep + 1}/{epochs}  val acc {acc:.3f} "
                  f"val loss {loss:.4f}")
            if acc > best_acc or (acc == best_acc and loss < best_loss):
```

```
                best_acc, best_loss, best_epoch = acc, loss, ep + 1
    return W, b, best_epoch


def evaluate_mlp(model, dataset, batch=512):
    loader = DataLoader(dataset, batch_size=batch)
    correct, total_loss, count = 0, 0.0, 0
    model.eval()
    with torch.no_grad():
        for images, labels in tqdm(loader, desc="[B] validation", unit="batch"):
            logits = model(images)
            correct += int((logits.argmax(1) == labels).sum())
            total_loss += float(nn.functional.cross_entropy(
                logits, labels, reduction="sum"
            ))
            count += len(labels)
    return correct / count, total_loss / count


def train_mlp_directory(model, dataset, epochs, val=None, lr=0.01,
                        batch=256, schedule_epochs=25):
    """Run the four-step SGD loop over every selected directory image."""
    loader = DataLoader(dataset, batch_size=batch, shuffle=True,
                        generator=torch.Generator().manual_seed(0))
    opt = torch.optim.SGD(model.parameters(), lr=lr, momentum=0.9)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
        opt, T_max=schedule_epochs, eta_min=lr * 0.1
    )
    loss_fn = nn.CrossEntropyLoss()
    best_acc, best_loss, best_epoch = -1.0, float("inf"), 1
    for ep in range(epochs):
        model.train()
        running_loss, seen = 0.0, 0
        with tqdm(loader, desc=f"[B] epoch {ep + 1}/{epochs}",
                  unit="batch") as progress:
            for images, labels in progress:
                opt.zero_grad()
                loss = loss_fn(model(images), labels)
                loss.backward()
                opt.step()
                running_loss += loss.item() * len(labels)
                seen += len(labels)
                progress.set_postfix(loss=f"{running_loss / seen:.4f}")
        if val is not None:
            acc, loss = evaluate_mlp(model, val)
            print(f"[B] epoch {ep + 1}/{epochs}  train loss "
                  f"{running_loss / seen:.4f}  val acc {acc:.3f} "
                  f"val loss {loss:.4f}")
            if acc > best_acc or (acc == best_acc and loss < best_loss):
                best_acc, best_loss, best_epoch = acc, loss, ep + 1
```

```
        scheduler.step()
    return best_epoch


def run_directory(dataset_path, linear_epochs=20, mlp_epochs=25):
    """Select epochs on held-out captures, then refit on all augmented images."""
    dataset_path = Path(dataset_path)
    with (dataset_path / "labels.json").open(encoding="utf-8") as stream:
        metadata = json.load(stream)
    train_indices, val_indices, val_groups = grouped_holdout(metadata)
    counts = np.bincount(
        np.fromiter((item["label"] for item in metadata), dtype=np.int64),
        minlength=N_CLASSES,
    )
    print(f"found {len(metadata)} images ({dict(enumerate(counts.tolist()))})")
    print(f"held out {len(val_groups)} complete captures: "
          f"{len(train_indices)} train, {len(val_indices)} validation images")
    train = CopybookDirectory(dataset_path, metadata, train_indices)
    validation = CopybookDirectory(dataset_path, metadata, val_indices)
    full = CopybookDirectory(dataset_path, metadata)

    _, _, linear_best_epoch = train_linear_directory(
        train, linear_epochs, val=validation, schedule_epochs=linear_epochs
    )
    print(f"[A] refitting on all {len(full)} images for {linear_best_epoch} epochs")
    W, b, _ = train_linear_directory(
        full, linear_best_epoch, schedule_epochs=linear_epochs
    )
    export_linear(W, b)

    sample, _ = full[0]
    XB = part_b_features(sample.numpy()[None, 0])
    assert XB.ndim == 2 and np.abs(XB).max() <= 1.0 + 1e-5, \
        "representation must be a bounded [n, d] matrix"
    torch.manual_seed(0)
    model = MLP()
    model.apply(init_weights)
    n_params = count_params(model)
    assert n_params <= BUDGET, f"{n_params} params exceeds the {BUDGET} budget"
    print(f"[B] representation {XB.shape[1]} dims; {n_params} trainable parameters")
    mlp_best_epoch = train_mlp_directory(
        model, train, mlp_epochs, val=validation, schedule_epochs=mlp_epochs
    )
    print(f"[B] refitting on all {len(full)} images for {mlp_best_epoch} epochs")
    torch.manual_seed(0)
    model = MLP()
    model.apply(init_weights)
    train_mlp_directory(model, full, mlp_best_epoch, schedule_epochs=mlp_epochs)
    export_mlp(model)
```

```
    import onnx
    graph = onnx.load("mlp_model.onnx")
    onnx.checker.check_model(graph)
    dims = [dim.dim_value for dim in graph.graph.input[0].type.tensor_type.shape.dim]
    assert dims == [1, 1, 64, 64], f"input shape {dims} — export block was modified?"
    print("[B] mlp_model.onnx verified")


# ---------------------------------------------------------------- run ---
# Runtime -> Run all. Fill TODOs in order; every assert pinpoints the next
# gap. Exported files are checked locally, then submit at /board.html.

if __name__ == "__main__":
    import argparse
    ap = argparse.ArgumentParser()
    ap.add_argument("--dataset", default="copybook.zip", help="your Copybook export")
    ap.add_argument("--val", default=None, help="optional manifest dir, e.g. the public set")
    ap.add_argument("--linear-epochs", type=int, default=20)
    ap.add_argument("--epochs", type=int, default=25)
    args, _unknown = ap.parse_known_args()   # Colab injects -f kernel.json; ignore it

    if Path(args.dataset).is_dir():
        if args.val:
            ap.error("directory training uses its own grouped validation split")
        run_directory(args.dataset,
                      linear_epochs=args.linear_epochs, mlp_epochs=args.epochs)
        raise SystemExit(0)

    images, labels = load_copybook(args.dataset)
    print(f"loaded {len(images)} traced images "
          f"({dict(zip(*np.unique(labels, return_counts=True)))})")

    # ---- Part A ------------------------------------------------------------
    XA = part_a_features(images)
    W, b = part_a_train(XA, labels)
    export_linear(W, b)

    # ---- Part B ------------------------------------------------------------
    XB = part_b_features(images)                        # TODO-B1 prototype
    assert XB.ndim == 2 and np.abs(XB).max() <= 1.0 + 1e-5, \
        "representation must be a bounded [n, d] matrix"
    print(f"[B] representation prototype: {XB.shape[1]} dims "
          f"(mirror the SAME preprocessing inside MLP.forward, TODO-B3)")
    if args.val:                                         # public set as validation
        vm = json.load(open(f"{args.val}/manifest.json"))
        vi = np.stack([np.asarray(Image.open(f"{args.val}/{m['file']}").convert('L'),
                                  np.float32) / 255.0 for m in vm])
        vl = np.array([m["label"] for m in vm], np.int64)
    else:
        cut = max(8, len(images) // 5)
```

```
    vi, vl = images[:cut], labels[:cut]

model = MLP()                                       # TODO-B3
model.apply(init_weights)                           # TODO-B4
n_params = count_params(model)                      # TODO-B2
assert n_params <= BUDGET, f"{n_params} params exceeds the {BUDGET} budget"
print(f"[B] architecture accepted: {n_params} parameters "
      f"({100 * n_params / BUDGET:.0f}% of budget)")

part_b_train(model, images, labels, vi, vl)         # TODO-B5
export_mlp(model)                                   # provided

import onnx
m = onnx.load("mlp_model.onnx")
onnx.checker.check_model(m)
dims = [d.dim_value for d in m.graph.input[0].type.tensor_type.shape.dim]
assert dims == [1, 1, 64, 64], f"input shape {dims} — export block was modified?"
print("[B] mlp_model.onnx verified — submit at /board.html and keep the "
      "submission ID for your report")
```